Panorama do cgu-bd

Banco local e workflow de engenharia reversa para o concurso CGU/AFFC 2026 (TI – Desenvolvimento de Sistemas)

Versão v4.5 | atualizado em 10/11/2026

Ideia central. O Claude não guarda o andamento na memória: o andamento fica em arquivos na sua máquina. Tudo que é mecânico (baixar, extrair, contar, comparar, ranquear, validar) roda localmente em Python e custa zero de créditos. O Claude entra só onde há julgamento: classificar itens, checar uma amostra e escrever aulas e cadernos.

Como manter este documento. Ao mudar qualquer coisa na estrutura, edite os blocos de DADOS no início de `docs/gerar_panorama.py` (principalmente o CHANGELOG) e rode `python3 docs/gerar_panorama.py`. Gerar o PDF é local e não gasta créditos.

1. O que construímos

O objetivo é descobrir **o que a banca mais cobra** em cada tema do edital e ensinar nessa ordem. Para isso montamos um banco de questões reais de provas anteriores, classificamos cada item por tema do edital, por *núcleo* (a ideia única que o item testa) e por tipo de pegadinha, e calculamos a massa de cada núcleo (peso da prova x recência). A aula ensina os núcleos até cobrir cerca de 80% da massa acumulada; o caderno usa só itens reais, com referência e URL.

Estado de cada prova (manifesto provas.jsonl). Cada seta só passa por um PORTÃO no banco.py.

Falha em qualquer portão: o estado não avança. Verificação reprovada: a prova volta para 'extraida'.

roda na sua máquina (sem créditos) usa o Claude (créditos)

Peças principais

Peça	Papel
banco.py	Programa único. Guarda o estado, aplica os portões, ranqueia e valida. Python padrão; precisa do pdftotext (ou do pypdf) para ler PDF.
Skills (3)	Manuais em .md carregados sob demanda: classificar itens, coletar questões, priorizar temas.
Agentes (2)	Executores com instruções próprias: classificador-itens e verificador-classificacao. Não ficam registrados na conta; o Claude lê o .md e os dispara.
Arquivos .jsonl	O estado: uma linha de JSON por registro. Abrem em qualquer editor.

2. Quem faz cada etapa e quanto custa

Verde = sua máquina, zero de créditos. Amarelo = Claude. A coluna de custo é qualitativa (relativa entre as etapas), não uma medida em tokens.

Etapa	Onde roda	Custo	Como
Importar o edital (PDF) para programa.json	**Máquina**	zero	scripts/importar_edital.py (uma vez por edital)
Ver subitens do programa sem questão	**Máquina**	zero	edital lacunas
Escolher a próxima prova a minerar	**Claude**	baixo	Lê o ranking de compatibilidade; depois 'prova add' (local)
Baixar PDF da prova e do gabarito	**Máquina**	zero	prova processar ID (Chrome só se o site bloquear)
Guardar PDFs, hash e origem	**Máquina**	zero	provas/ID/ + fonte.txt
Extrair texto, gabarito e itens	**Máquina**	zero	pdftotext + extrator no banco.py
Portão: itens = gabaritos	**Máquina**	zero	recusa avançar se não bater
Conferir itens marcados 'revisar'	**Claude**	baixo	só os poucos itens com código ou figura
Classificar itens (edital, núcleo, pegadinha)	**Claude**	ALTO	agente classificador-itens, 1 por prova
Sortear amostra de 20%	**Máquina**	zero	prova amostra ID
Reclassificar a amostra (checagem independente)	**Claude**	médio	agente verificador-classificacao
Comparar concordância e decidir	**Máquina**	zero	comparar ID (mínimos 90/80/80)
Gravar no banco	**Máquina**	zero	aplicar ID
Ranking, cobertura e plano da aula	**Máquina**	zero	rank --edital ...
Escrever a aula (PDF dark mode)	**Claude**	ALTO	explicação do zero, ilustrações, como a banca cobra
Escolher os ids do caderno (máx. 10)	**Claude**	baixo	a partir do banco
Validar o caderno	**Máquina**	zero	verificar-caderno lista.json
Atualizar este panorama (PDF)	**Máquina**	zero	python3 docs/gerar_panorama.py

Nota honesta. Mesmo as etapas locais são disparadas pelo Claude: cada comando é uma chamada curta e devolve uma linha de resumo. A economia vem de o trabalho pesado (PDF, texto, contagens, comparações) não entrar na conversa.

3. Onde a economia acontece

Tarefa	Antes	Agora
Download e leitura de PDF	Navegador do Claude e leitura do conteúdo na conversa	Download direto pela máquina; o PDF nunca entra na conversa
Separar itens e gabarito	O Claude lia o texto e montava os itens	Extrator em Python; devolve só a contagem e os itens a revisar
Conferir contagens, hash e cobertura	Raciocínio do Claude a cada vez	Portões no banco.py (determinísticos)
Verificar a classificação	Reclassificar tudo	Amostra de 20% e comparação automática
Ranking e plano da aula	Cálculo manual em prosa	rank --edital: calculado localmente
Validar o caderno	Conferir id por id na conversa	verificar-caderno devolve APROVADO ou a lista de problemas

Onde os créditos ainda são gastos (e é correto que sejam)

1) Classificar itens é o maior custo recorrente, proporcional ao número de itens. **2) Escrever aulas**, que exigem explicação do zero e material visual. **3) Verificar a amostra**, cerca de 20% do custo de uma segunda classificação. Não existe modelo local que substitua esse julgamento com a mesma qualidade.

Regras para manter o gasto baixo

• Passar ao agente só o **caminho** do arquivo, nunca o conteúdo.
• Não abrir prova.txt, exceto para consertar item sinalizado.
• Um agente por prova, devolvendo só a contagem e os itens com dúvida.
• Não reclassificar item que já está no banco.
• Classificar só provas compatíveis com o edital (ranking de compatibilidade).
• Reservar o Chrome para quando o download direto falhar.

Ideias ainda não aplicadas

• Definir um modelo mais leve no campo *model* do agente classificador e medir a concordância com o próprio *comparar* antes de adotar.
• Filtrar por palavras-chave os itens claramente fora do edital antes de classificar.
• Agrupar a classificação em lotes por tema.

4. Estrutura de pastas e portões

Caminho	Conteúdo
README.md / ORQUESTRACAO.md	Entrada e roteiro que o Claude lê primeiro
banco.py / scripts/	Programa único (Python): estado, portões, ranking, validações; importar_edital.py lê o PDF do edital
concursos/ativo.txt	Qual concurso está em uso (troca com: banco.py concurso usar ID)
concursos//	concurso.json (cargo-alvo), edital-oficial.pdf, programa.json/.md, regras-da-prova.md e os dados do concurso: banco, núcleos, manifesto, classificações
provas/ID/	PDFs da prova e do gabarito, fonte.txt, prova.txt, gabarito.txt
extraidos/ (compartilhado)	Itens extraídos de cada prova, um .jsonl por prova
skills/	Manuais: classificação, coleta, priorização (cópia das skills da conta)
agentes/	Instruções dos 2 agentes (classificador e verificador)
edital-topicos.md / taxonomia-pegadinhas.md	Códigos do edital e etiquetas de pegadinha
docs/	Manuais: ARQUIVOS, workflow-v2, portabilidade, regras do Projeto e este PANORAMA
referencia/	Documentos históricos transcritos do Projeto

Portões (o banco.py recusa avançar se falhar)

Estado	Condição
baixada	provas/ID/prova.pdf e gabarito.pdf válidos; hash gravado (ou --sem-arquivo com nota)
extraida	itens extraídos = gabaritos (anulados fora) = linhas de extraidos/ID.jsonl
classificada	classificacoes/ID.jsonl cobre exatamente os itens; edital e etiquetas válidos
verificada	só por 'comparar': concordância mínima de 90% (edital), 80% (núcleo) e 80% (pegadinha)
persistida	só por 'aplicar'

Comandos mais usados

`prova add ...` cadastra a prova. `prova processar ID` baixa, extrai e avança. `prova proxima` mostra o que falta. `prova amostra ID`, `comparar ID` e `aplicar ID` fecham a verificação. `rank --edital "P3·ENG 2"` planeja a aula. `verificar-caderno lista.json` valida o caderno. Todos começam com `python3 banco.py`.

5. Pendências e limites

• Classificar a prova TCE-AC 2024 cargo 11 (70 itens extraídos, estado 'extraida'). Cobertura do programa hoje: 1 de 205 subitens com questão.

• Banco, núcleos e classificações são por concurso; provas/ e extraidos/ são compartilhados. Rank ainda usa limiares fixos no código (concurso.json guarda os valores, mas o rank não os lê).

• Extrator testado só no layout Cebraspe de duas colunas; outras bancas podem exigir ajuste.

• Itens 70 e 97 da TCE-AC marcados 'revisar' (código no enunciado).

• O espelho no Projeto do Claude está desatualizado em relação ao PC (a fonte correta é o PC).

• Limiares (70% de cobertura, 80% de massa, 90/80/80 de concordância) são regras iniciais a calibrar.

• Termos de uso do TEC Concursos ainda não lidos; o TEC bloqueou acessos (405/429).

6. Histórico de versões

Data	Versão	Mudança
10/11/2026	v4.6	Auditoria ponderada das amostras cegas estratificadas P2/P3: precisão e IC90 reproduzíveis; recall explicitamente não estimável sem amostra representativa dos descartados.
10/11/2026	v4.5	Adicionadas filas de tarefas ML locais sem aplicação automática, validação estrutural dos lotes e pendências explícitas de revisão semântica. O banco só avança pelos portões de classificação/verificação existentes.
10/11/2026	v4.4	Mineração local recursiva com blacklist por URL/hash, retomada de downloads válidos, interrupção explícita em 403/429 e gates de confiança por bloco. P2/P3 ficam provisórios até validação independente.
04/11/2026	v4.2	Pasta concursos// com concurso.json (cargo-alvo), edital oficial, programa.json/.md (275 subitens) e regras da prova. banco.py: concurso ativo, edital lacunas/buscar, portão valida códigos contra o programa. scripts/importar_edital.py.
04/11/2026	v4.1	Download, texto, gabarito e itens passaram a rodar local (prova baixar/texto/extrair/processar). Pasta provas/ID/ guarda os PDFs. Pasta docs/ criada. Este PDF criado.
04/11/2026	v4.0	banco.py com manifesto de provas, portões, comparar, aplicar, rank e verificar-caderno. Agentes classificador e verificador. Skill de classificação.
02/11/2026	v3	Banco local inicial (75 itens), núcleos, taxonomia de pegadinhas, workflow probabilístico.